

HORUS-LAB · ÉQUIPE TECHNIQUE

Bien utiliser Claude Code

Comment tirer parti d'un assistant de développement sans perdre la maîtrise de son code, de son dépôt, ni de la paternité de son travail.

HORUS-LAB

Solutions technologiques intelligentes

Version 1 · Juillet 2026

Document interne — équipe de développement

1. Le principe : qui décide ?
2. Installation et configuration
3. Attribution : le code est le vôtre
4. Le fichier de contexte du projet
5. Formuler une bonne demande
6. Le cycle de travail en 5 temps
7. Commandes et raccourcis utiles
8. Savoir dire non
9. Relire : la règle du diff
10. Sécurité et permissions
11. Claude Code et Git
12. Pièges fréquents
13. Checklist qualité

Le principe : qui décide ?

Claude Code est un **outil**, au même titre qu'un compilateur ou un débogueur. Il rédige vite et lit beaucoup de fichiers, mais il n'a ni la responsabilité du produit, ni la connaissance du métier, ni le dernier mot.

✓ Ce qu'il fait très bien

- Explorer un code inconnu et le résumer
- Écrire du code répétitif, des tests, de la doc
- Refactoriser sur plusieurs fichiers
- Expliquer une erreur, proposer des pistes
- Automatiser des tâches mécaniques

✗ Ce qu'il ne remplace pas

- Vos décisions d'architecture
- La connaissance du besoin client
- Les tests réels sur l'application
- La revue de code entre humains
- Votre responsabilité sur ce qui part en production

La règle Horus-Lab

Le développeur reste **l'auteur et le responsable** de tout ce qu'il pousse. Utiliser un assistant ne se délègue pas : ce que vous validez, vous l'assumez.

2 Installation et configuration

```
# Installation (Node 18+)
npm install -g @anthropic-ai/claude-code

# Lancement, depuis la racine du projet
cd website-horus-lab
claude
```

Les trois niveaux de configuration

Fichier	Portée
~/.claude/settings.json	Vous , sur toutes vos machines et tous vos projets
.claude/settings.json	L'équipe — versionné dans le dépôt, s'applique à tous
.claude/settings.local.json	Vous , sur ce projet seulement — non versionné

À ignorer dans Git

Le dossier `.claude/` contient de l'état local (sessions, worktrees). Il doit figurer dans le `.gitignore` — sinon il finit dans un commit, parfois avec un dépôt imbriqué qui casse les clones.

3 Attribution : le code est le vôtre

Par défaut, l'outil ajoute une ligne `Co-Authored-By` à chaque commit. Résultat : sur GitHub, un contributeur « Claude » apparaît dans le dépôt, ce qui **dilue le travail de l'équipe**. Chez Horus-Lab, on le désactive.

```
// ~/.claude/settings.json - à faire sur CHAQUE machine
{
  "includeCoAuthoredBy": false
}
```

Pour l'imposer à toute l'équipe d'un coup, la même clé peut être placée dans `.claude/settings.json`, versionné dans le dépôt.

Où la mention peut apparaître	Comment l'éviter
Message de commit	<code>includeCoAuthoredBy: false</code>
Description de Pull Request	Relire et retirer le pied de page avant de créer la PR
Commentaires dans le code	Ne jamais laisser passer un commentaire signé
Historique déjà poussé	Réécriture d'historique — opération lourde, voir §11

Ce n'est pas de la dissimulation

C'est une règle d'attribution d'entreprise, comme pour un IDE ou un générateur de code : l'outil ne cosigne pas le travail de l'équipe. En revanche, on n'ement pas sur la qualité : le code est relu et testé, donc assumé.

4 Le fichier de contexte du projet

Un fichier `CLAUDE.md` (ou `AGENTS.md`) à la racine décrit le projet. Il est lu automatiquement à chaque session : c'est le meilleur investissement pour obtenir des réponses justes.

```
# Monorepo Horus-Lab
frontend/  Next.js (site public)
backend/   Django REST API (CMS)

## Règles
- Commandes frontend depuis `frontend/`, backend depuis `backend/`
- Le contenu affiché vient du CMS Django, pas des fallbacks
- Toujours vérifier `npm run build` avant de proposer un commit
- Ne jamais modifier `.env` ni les fichiers de déploiement
```

Un bon fichier de contexte contient : la **structure**, les **commandes** (build, test, lint), les **conventions** et surtout les **pièges maison**. Mettez-le à jour dès qu'une règle change.

La qualité du résultat dépend directement de la précision de la demande. Une bonne demande contient un **objectif**, un **contexte** et des **contraintes**.

✗ Demande floue

- « Améliore le site »
- « Corrige les bugs »
- « Fais un truc plus joli »

✓ Demande exploitable

- « Dans `Header.tsx`, le menu mobile doit s'ouvrir en panneau latéral depuis la droite, sur la moitié de l'écran, sans toucher au menu desktop. »

Trois réflexes qui changent tout

- **Désigner les fichiers** avec `@chemin/fichier.tsx` plutôt que de décrire vaguement.
- **Donner le critère de réussite** : « le build doit passer », « sans changer l'API ».
- **Demander un plan d'abord** sur les tâches lourdes, et le valider avant l'écriture.

Une tâche à la fois

Une demande contenant huit sujets produit huit travaux moyens. Découpez : vous relirez mieux, et vous pourrez annuler un point sans perdre les autres.

6 Le cycle de travail en 5 temps

Étape	Ce que vous faites
1. Explorer	Demandez d'abord une <i>compréhension</i> du code concerné, sans modification.
2. Planifier	Faites décrire l'approche et les fichiers touchés. Corrigez le tir maintenant : c'est gratuit.
3. Écrire	Laissez implémenter, par petits lots plutôt qu'en une fois.
4. Vérifier	Build, lint, test, puis l'application réelle dans le navigateur.
5. Committer	Vous relisez le diff, vous écrivez ou validez le message, vous poussez.

L'erreur la plus coûteuse

Enchaîner « fais ça », « fais ci » pendant une heure sans jamais relire ni tester, puis committer 40 fichiers d'un bloc. Personne — ni vous, ni le relecteur — ne sait plus ce qui a changé.

7 Commandes et raccourcis utiles

Commande	Effet
<code>/init</code>	Génère un premier fichier de contexte pour le projet
<code>/clear</code>	Repart d'une conversation vide — à faire entre deux tâches sans lien
<code>/help</code>	Liste les commandes disponibles
<code>/model</code>	Change de modèle selon la difficulté de la tâche
<code>/config</code>	Ouvre les réglages
<code>/permissions</code>	Gère ce que l'outil a le droit d'exécuter
<code>@fichier</code>	Référence un fichier précis dans votre message
<code>!commande</code>	Exécute une commande shell et injecte sa sortie
Échap	Interrompt immédiatement le travail en cours

Le réflexe `/clear`

Une session qui traîne accumule du contexte inutile et rend les réponses moins pertinentes. Nouvelle tâche = nouvelle session.

8 Savoir dire non

Un assistant propose avec assurance, y compris quand il se trompe. Refuser une proposition est un acte technique normal — c'est même le cœur du métier.

Quand refuser	Quoi répondre
Il réécrit un fichier entier pour un détail	« Fais la modification minimale, ne touche pas au reste. »
Il ajoute une dépendance non justifiée	« Sans nouvelle dépendance : fais-le avec l'existant. »
Il invente une fonction ou une option	« Vérifie dans le code / la doc avant d'affirmer. »
Il change une décision d'architecture	« Cette structure est volontaire, garde-la. »
Il dit « c'est corrigé » sans preuve	« Montre-moi le test qui passe. »
La solution vous paraît trop compliquée	« Propose une approche plus simple. »

Signal d'alerte

Si vous ne comprenez pas le code produit, **ne le committez pas**. Demandez une explication, ou une version plus simple. Du code non compris est une dette immédiate.

9 Relire : la règle du diff

La relecture n'est pas optionnelle. Avant tout commit :

```
# Ce qui a changé, fichier par fichier
git status
git diff

# Puis, seulement si tout est compris :
git add -p      # ajoute morceau par morceau, en relisant
```

- Cherchez les **fichiers hors sujet** : s'ils n'ont rien à voir avec la tâche, écartez-les.
- Traquez les **suppressions** non demandées : c'est là que se perdent des fonctionnalités.
- Vérifiez les **valeurs en dur**, les identifiants, les URL.
- Méfiez-vous des **gros fichiers binaires** ajoutés sans raison.

Astuce

`git add -p` vous force à voir chaque morceau avant de l'accepter. C'est le meilleur filet de sécurité contre un commit involontaire.

10 Sécurité et permissions

Interdit	À la place
Coller un mot de passe, un token ou une clé dans la conversation	Utiliser des variables d'environnement, et ne jamais les afficher
Laisser exécuter des commandes sur le serveur de production	Faire tourner les commandes vous-même, en connaissance de cause
Autoriser en masse toutes les commandes	Autoriser au cas par cas, via /permissions
Laisser committer et pousser sans relecture	Relire le diff, puis décider

Rappel

Tout ce qui est écrit dans une conversation quitte votre machine. Un secret partagé une fois est un secret à révoquer.

- Travaillez **toujours sur une branche**, jamais directement sur `main`.
- Committez **vous-même**, ou relisez intégralement le commit proposé.
- Vérifiez le message : il doit respecter la convention d'équipe et ne pas mentionner l'outil.
- Ne laissez jamais faire un `push --force` sur une branche partagée.

Nettoyer un historique déjà pollué

Si des commits déjà poussés contiennent la mention `Co-Authored-By`, seule une **réécriture d'historique** peut l'effacer. C'est une opération lourde :

- tous les identifiants de commit changent ;
- toute l'équipe doit **re-cloner** le dépôt ;
- les Pull Requests ouvertes sont cassées — il faut les fermer ou les fusionner avant ;
- les descriptions de PR et les commentaires GitHub **ne sont pas** modifiés : ils se nettoient à la main.

À décider une seule fois

Faites-le tôt, quand l'équipe est petite, ou pas du tout. Le plus important reste de couper la source :
`includeCoAuthoredBy: false` partout, dès aujourd'hui.

12 Pièges fréquents

Piège	Conséquence et parade
Accepter sans lire	Du code non maîtrisé en production. Parade : relire le diff, toujours.
Demande trop large	Un travail moyen partout. Parade : découper.
Session trop longue	Réponses moins pertinentes. Parade : <code>/clear</code> .
Croire « c'est corrigé »	Le bug est toujours là. Parade : exiger une vérification.
Contexte projet absent	Réponses hors conventions. Parade : tenir le <code>CLAUDE.md</code> à jour.
Ajouts de fichiers parasites	Dépôt pollué. Parade : <code>git status</code> avant chaque <code>add</code> .
Dépendance ajoutée « au passage »	Surface d'attaque et poids inutiles. Parade : refuser par défaut.

13 Checklist qualité

✓	Avant de committer un travail assisté
☐	J'ai lu tout le diff et je comprends chaque ligne
☐	Le projet compile et j'ai testé le comportement réel
☐	Aucun fichier hors sujet, aucun secret, aucun dossier local
☐	Aucune dépendance ajoutée sans décision explicite
☐	Le message de commit suit la convention et ne cite aucun outil
☐	La description de PR est écrite pour un humain
☐	Je suis capable d'expliquer et de défendre ce code en revue

L'esprit du guide

Bien utiliser un assistant, ce n'est pas taper moins : c'est **décider mieux**. L'outil accélère l'exécution, l'ingénieur garde le jugement. C'est cette maîtrise qui fait la valeur des développeurs de Horus-Lab.